

Introduction to MPI

Frederick Fung

National Computational Infrastructure, Australia

Prepared based on MPI Standard 4.0

Acknowledgement of Country

The National Computational Infrastructure acknowledges, celebrates and pays our respects to the Ngunnawal and Ngambri people of the Canberra region and to all First Nations Australians on whose traditional lands we meet and work, and whose cultures are among the oldest continuing cultures in human history.

Disclaimer

This work represents the view of the author delivering this workshop.

Goal

This two half-day workshop consists of lectures and coding exercises. The goal of this session is to get users familiar with a broad range of common communication mechanisms offered by the MPI library.

Goal

This two half-day workshop consists of lectures and coding exercises. The goal of this session is to get users familiar with a broad range of common communication mechanisms offered by the MPI library.

Caveat

Lower your expectations. You are unlikely becoming an MPI expert after studying one code example.
(Use LLM of your choice to help write an expert MPI code)

● Day 1

- ▶ Syntax and semantics
- ▶ Point-to-point communication
 - ★ Blocking communication
 - ★ Non-blocking communication
 - ★ Overlapping communication with computation
 - ★ Persistent communication

● Day 2

- ▶ Collective communication
- ▶ One-sided communication
- ▶ MPI Profiling
- ▶ MPI-IO (optional)

● Day 1

- ▶ Syntax and semantics
- ▶ Point-to-point communication
 - ★ Blocking communication
 - ★ Non-blocking communication
 - ★ Overlapping communication with computation
 - ★ Persistent communication

● Day 2

- ▶ Collective communication
- ▶ One-sided communication
- ▶ MPI Profiling
- ▶ MPI-IO (optional)

In this workshop, we will demonstrate each of the communication patterns in a two-dimensional finite difference method example.

- MPI stands for Message Passing Interface.
 - ▶ The development of the standardisation is community driven and is derived from state-of-art research by university researchers, vendors of hardware, government laboratories and industry.

¹The US Department of Energy Office of Science and the National Nuclear Security Administration initiated the Exascale Computing Project (ECP) in 2016 to prepare mission-relevant applications and scientific software for the delivery of the exascale computers starting in 2023.

- MPI stands for Message Passing Interface.
 - ▶ The development of the standardisation is community driven and is derived from state-of-art research by university researchers, vendors of hardware, government laboratories and industry.
 - ▶ There are many implementations of the standard: MPICH, OpenMPI, Intel MPI, Cray MPI, IBM MPI etc.
- MPI is suitable for use by MIMD (multiple instruction multiple data) or SPMD (single program multiple data) programs.
 - ▶ Such program usually runs on distributed-memory multiprocessors.

¹The US Department of Energy Office of Science and the National Nuclear Security Administration initiated the Exascale Computing Project (ECP) in 2016 to prepare mission-relevant applications and scientific software for the delivery of the exascale computers starting in 2023.

- MPI stands for Message Passing Interface.
 - ▶ The development of the standardisation is community driven and is derived from state-of-art research by university researchers, vendors of hardware, government laboratories and industry.
 - ▶ There are many implementations of the standard: MPICH, OpenMPI, Intel MPI, Cray MPI, IBM MPI etc.
- MPI is suitable for use by MIMD (multiple instruction multiple data) or SPMD (single program multiple data) programs.
 - ▶ Such program usually runs on distributed-memory multiprocessors.
- MPI dominates HPC programs [?].
 - ▶ All responded projects in Exascale Computing Projects¹ use MPI.

¹The US Department of Energy Office of Science and the National Nuclear Security Administration initiated the Exascale Computing Project (ECP) in 2016 to prepare mission-relevant applications and scientific software for the delivery of the exascale computers starting in 2023.

A brief history of the MPI standard

- More than 30 years of MPI: the first standard appeared in 1994.
 - ▶ 1992: Standardisation work began.
 - ▶ 1994: MPI 1.0 — point-to-point, datatypes and collectives.
 - ▶ 1995: MPI 1.1 — corrections and clarifications.
 - ▶ 1997: MPI 1.2 — clarifications, published with MPI 2.0.
 - ▶ 1997: MPI 2.0 — parallel I/O, RMA and dynamic processes.
 - ▶ 2008: MPI 1.3 consolidated MPI-1; MPI 2.1 unified MPI-1 and MPI-2.
 - ▶ 2009: MPI 2.2 — corrections and small additions.
 - ▶ 2012: MPI 3.0 — nonblocking collectives and extended RMA.
 - ▶ 2015: MPI 3.1 — clarifications and nonblocking collective I/O.
 - ▶ 2021: MPI 4.0 — large counts, persistent collectives, partitioned communication and Sessions.
 - ▶ 2023: MPI 4.1 — clarifications and minor extensions.
 - ▶ 2025: MPI 5.0 — standard Application Binary Interface (ABI).

Adapted from George Bosilca; updated for 2026.

Source: MPI Forum standards and release history.

- Scalability in parallel computing
 - ▶ Solving kN scale problem with kN computational resources during the same computation time - **weak scaling**.

- Scalability in parallel computing
 - ▶ Solving kN scale problem with kN computational resources during the same computation time - **weak scaling**.
 - ▶ Solving a problem with kN computational resources during $1/k$ computation time - **strong scaling**.

- Scalability in parallel computing
 - ▶ Solving kN scale problem with kN computational resources during the same computation time - **weak scaling**.
 - ▶ Solving a problem with kN computational resources during $1/k$ computation time - **strong scaling**.

MPI utilises a large number amount of computational resources available to users i.e. making the kN term.

On Gadi, two MPI implementations are available by Environment module.

- Intel MPI e.g. `$module load intel-compiler/[version]`, `$module load intel-mpi/[version]`
- OpenMPI e.g. `$module load openmpi/[version]`

Compilation:

- Fortran: `$mpi77`, `mpif90`
 - ▶ `$ mpif77`
 - ▶ `$ mpif90`
 - ▶ `$ mpiifort`
- C/C+:
 - ▶ `$ mpicc`
 - ▶ `$ mpiicc`

- MPI does

- ▶ provide you a communication library;
- ▶ provide you datatypes;
- ▶ have language bindings with C and Fortran;
- ▶ provide parallel I/O;
- ▶ provide interface for tool support;
- ▶ and many more.

- MPI does NOT

- ▶ design parallelism for your serial code.

- MPI does
 - ▶ provide you a communication library;
 - ▶ provide you datatypes;
 - ▶ have language bindings with C and Fortran;
 - ▶ provide parallel I/O;
 - ▶ provide interface for tool support;
 - ▶ and many more.
- MPI does NOT
 - ▶ design parallelism for your serial code.

Users are responsible for the correctness of the implementation of their parallel algorithms. We use the next example to elaborate.

Example: Monte-carlo Approximation of π

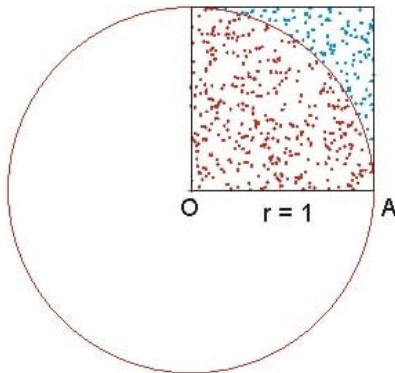


Figure 1: Generate N random sampling points within a square, and count the number h of samples that fall in the circle. Then the approximation $\pi \approx 4h/N$

A serial code

x, y are normalised random numbers. Variable declarations are not shown.

Parallel MPI code

Users organise the copies of data for each parallel processes.

Six Mostly Used MPI Calls

- MPI_INIT
- MPI_COMM_RANK
- MPI_COMM_SIZE
- MPI_SEND
- MPI_RECV
- MPI_FINALIZE

"Start" MPI Process

MPI_INIT

MPI_INIT()

C Binding

- The call does not create a process; rather, it initialises the MPI environment.
 - ▶ e.g defines the initial communicator `MPI_WORLD_COMM`.
- MPI functions must only be called after this call.

- Group of processes for communication.
- Communicator must be specified in MPI program as a unit of communication.
 - ▶ Default is `MPI_WORLD_COMM`.
- Multiple communicators can be defined, either overlapping or non-overlapping.

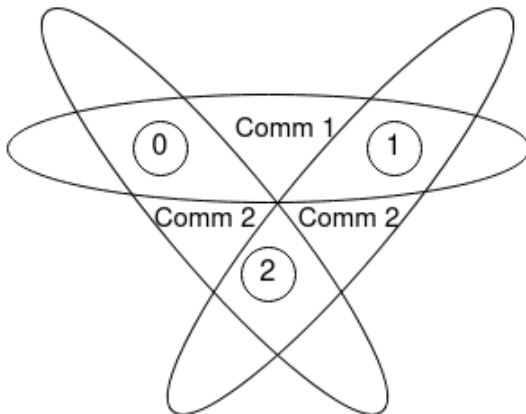


Figure 2: Overlapping communicators [?]

MPI_COMM_SIZE

MPI_COMM_SIZE(**comm**, **size**)

IN **comm** *communicator (handle)*

OUT **size** *number of processes in the group of comm (integer)*

C Binding

MPI_COMM_RANK

MPI_COMM_RANK(**comm**, **rank**)

IN **comm** *communicator (handle)*

OUT **rank** *rank of the calling process in group of comm (integer)*

C Binding

"Close" MPI Process

MPI_FINALIZE

MPI_FINALIZE()

C Binding

- This call does not shut down the processes.
 - ▶ Behaviour could be undefined if called on a process that still participates in communication.
 - ▶ Consider MPI_Abort to capture catastrophic errors.

The structure of a basic MPI program in C



Review the program MC_pi.c to understand the structure of a basic MPI program.

Consider a two-dimensional Poisson equation with Dirichlet boundary condition over a unit square domain $\Omega = [0, 1] \times [0, 1]$

$$-\Delta u = f \text{ in } \Omega$$

$$u = g \text{ on } \partial\Omega$$

Finite difference

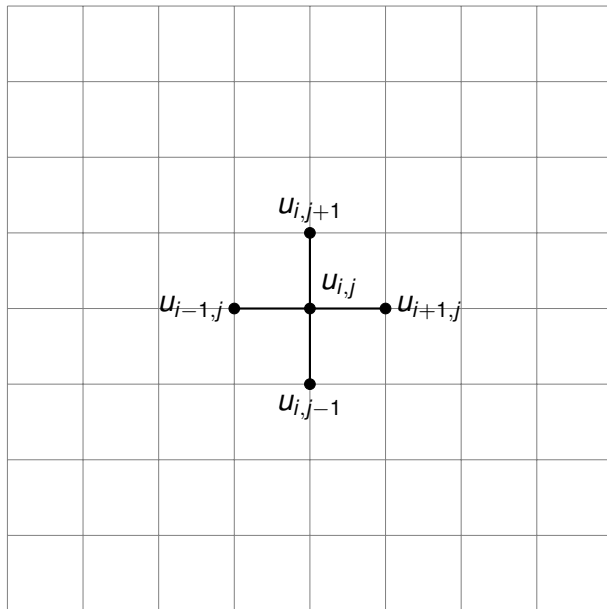
We use the second-order central finite difference method to discretise the Laplace operator

$$\begin{aligned}(\Delta u)_{i,j} &= (D_{xx}^2 u)_{i,j} + (D_{yy}^2 u)_{i,j} \\ &\approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}\end{aligned}$$

leading to

$$-(\Delta u)_{i,j} = \frac{4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1}}{h^2} = f(u_{i,j}).$$

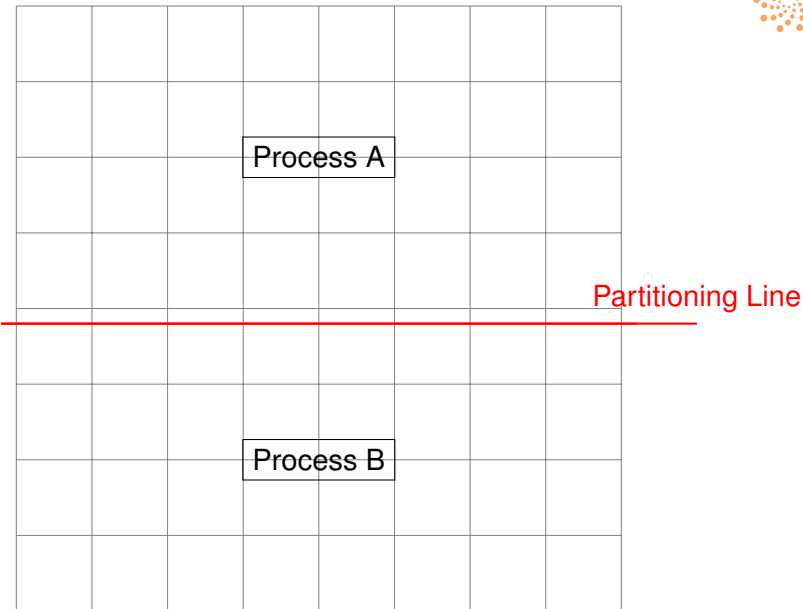
Model Problem: the mesh and the stencil



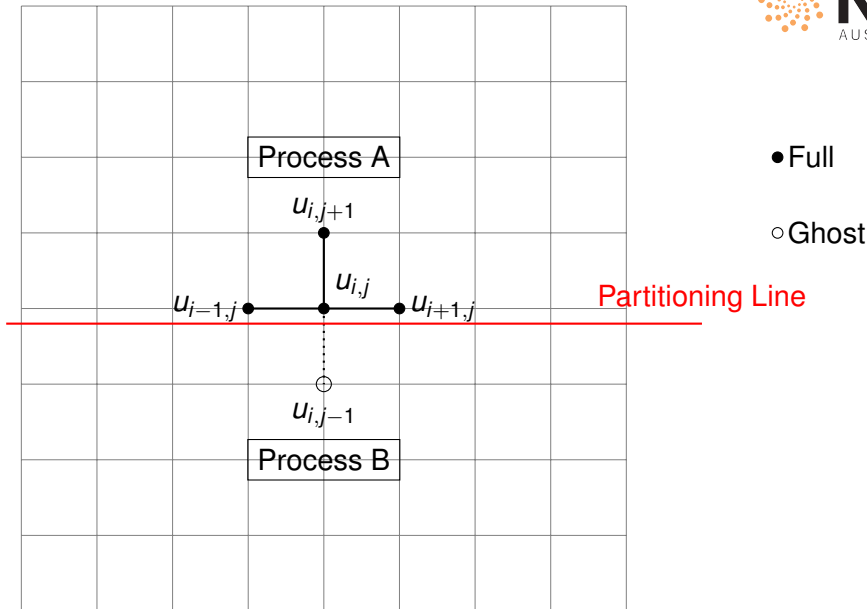
Domain decomposition

Users are responsible to decompose the mesh for parallelisation. For simplicity, the mesh is decomposed vertically in our practice code. Further, unless otherwise stated, we assume the mesh is split into upper submesh and the lower submesh and are distributed to two processes respectively.

Model Problem: Parallelisation



Model Problem: Ghost nodes vs Full nodes



Problem: How to exchange the ghost node information between Process A and B?

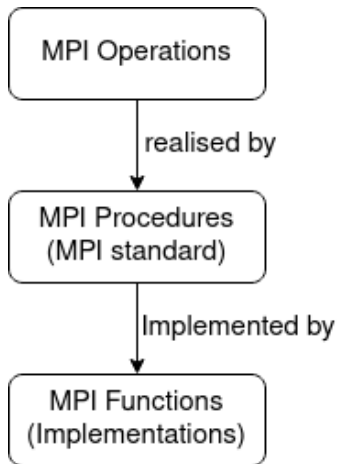
Topic 1: Syntax and Semantics

Topic 1: Syntax and Semantics

Warning: It does get tedious

- MPI Operations
- MPI Procedures
- MPI Functions

- MPI Operations
- MPI Procedures
- MPI Functions



Definition [?]

An MPI operation is a sequence of steps performed by the MPI library to establish and enable data transfer and/or synchronisation. It consists of four stages: **initialisation**, **starting**, **completion**, and **freeing**, and it is implemented as a set of one or more MPI procedures.

- Blocking operations
- Nonblocking operations
- Persistent operations
- Collective operations

Remark

An MPI operation describes the property of a class of data transfer mechanism but does not define a unique MPI procedure.

Definition Blocking operation

All four stages are combined in a single procedure call.

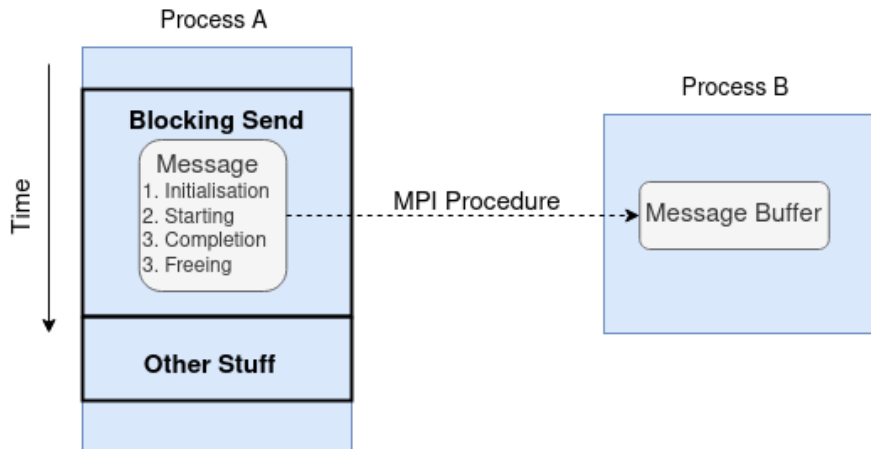
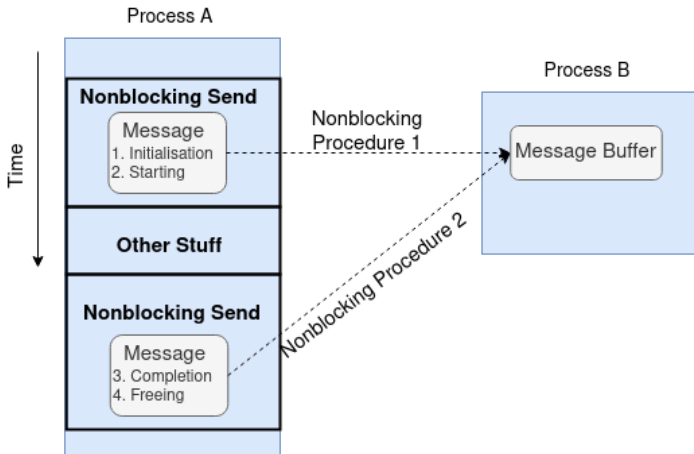


Figure 3: The send operation is blocking at process A

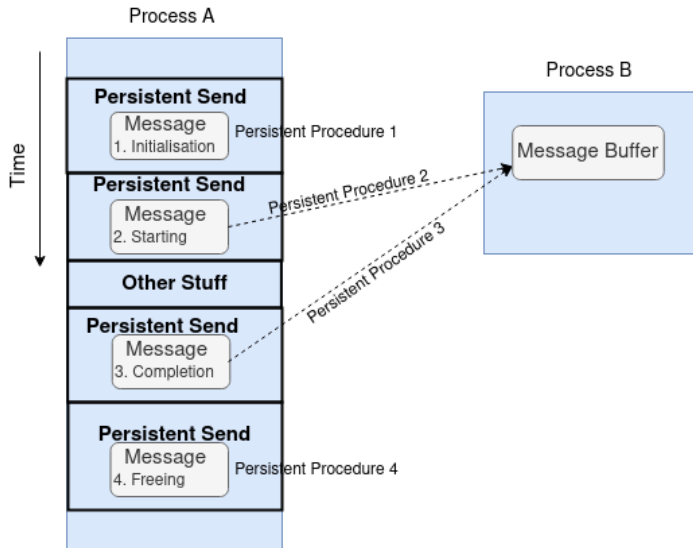
Definition Nonblocking operation

The initialization and starting stages are combined into a single nonblocking procedure call and the completion and freeing stages are combined into a separate, single procedure call.



Definition Persistent operation

There is a separate procedure for each of the four stages.



Definition

MPI procedures describe functionalities and are specified using a language-independent notation. An MPI operation-related procedure implements at least a part of a stage of an MPI operation.

- Examples

- ▶ MPI_SEND, MPI_PROBE (>400) (Note all letters are capitalised)

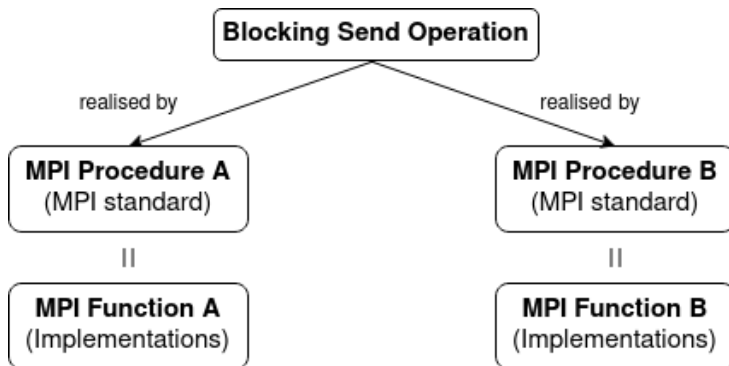
All MPI procedures can either be local or non-local: Whether its returning requires it to be called on another MPI Process.

Definition

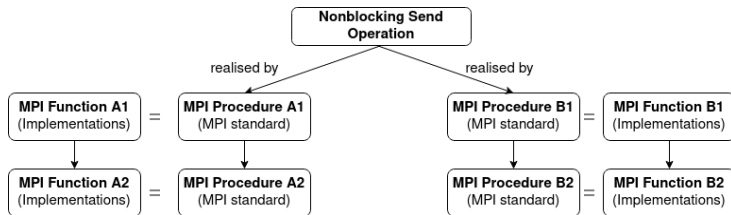
In this workshop, MPI functions are MPI procedures implemented by MPI implementations with language bindings.

- Example
 - ▶ MPI_Send, MPI_Probe etc

We summarise the semantics with the send operation example.



With the nonblocking send example.



Topic 2: Point-to-point Communication

What is a message?

Message = Message data + Message envelope

Message data

(address, count, datatype)

Information consists of count successive entries of the type indicated by datatype, starting with the entry at address buffer.

Message envelope

(source, destination, tag, communicator)

Information that can be used to distinguish messages and selectively receive them.

What is a message?

- *datatype*: A predefined name to specify machine-independent data elements.
 - ▶ Basic datatypes: MPI_INT, MPI_DOUBLE (in C); MPI_INTEGER, MPI_REAL (in Fortran).
 - ▶ Derived datatypes: mix of basic datatypes.
 - ▶ MPI datatype is not an actual datatype in the host language.
 - ▶ Data can be communicated between machines with different memory representations.
- *tag*: distinguishes different types of messages.
- *communicator*: specifies a "communication universe". Messages are always received within the "communication universe" they were sent, and messages sent in different "communication universe" do not interfere.
 - ▶ MPI_COMM_WORLD: communication within all processes available at initialisation time.

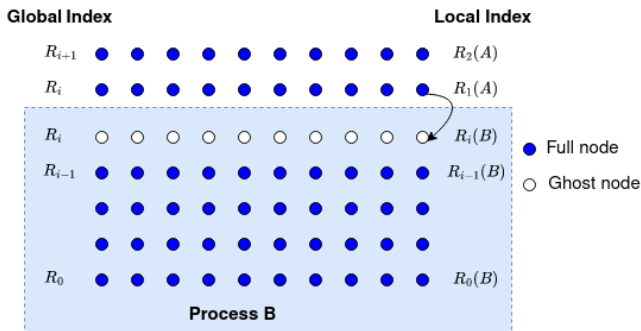


Figure 4: The message of row i is sent from Process A and received by Process B

- Blocking SEND-RECV is the basic MPI communication mechanism.

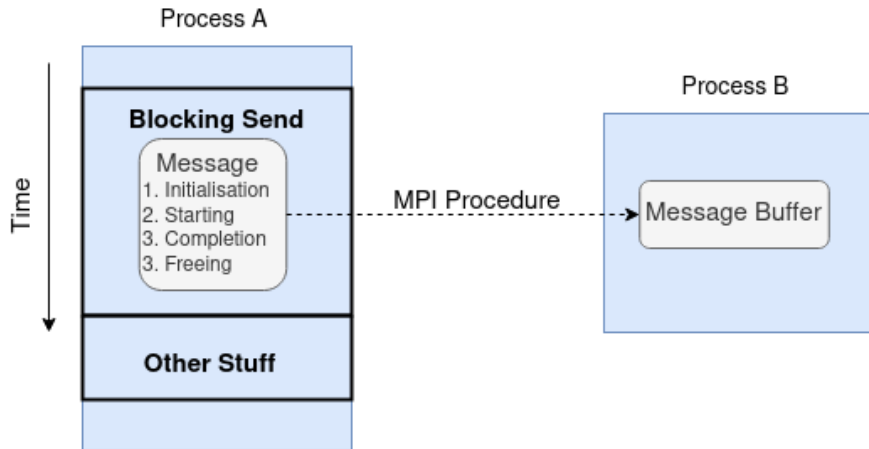


Figure 5: A simplified schematic diagram: the data message is sent from process A and received by process B through a communication network.

Definition 1. Blocking Operation

All four stages are combined in a single procedure call.

Definition 2. Blocking Send

The corresponding MPI procedure does not return until the message data and envelope have been safely stored away so that the sender is free to modify the send buffer.

Definition 1 $\implies$ Definition 2

Remark

For the blocking send, the message might be sent directly into the matching receiver's buffer or it might be copied into a temporary system buffer. MPI offers the following **communication modes** for users to choose how they want to complete the blocking.

- Standard Send
- Buffered Send
- Synchronous Send
- Ready Send

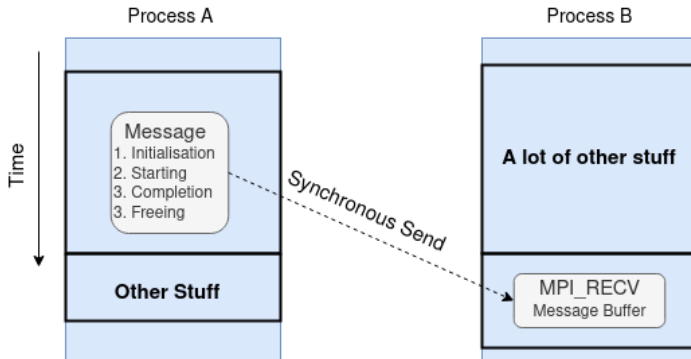


Figure 6: The SEND from processor A is held until the matching RECV is posted in processor B.

MPI_SEND (Standard mode)

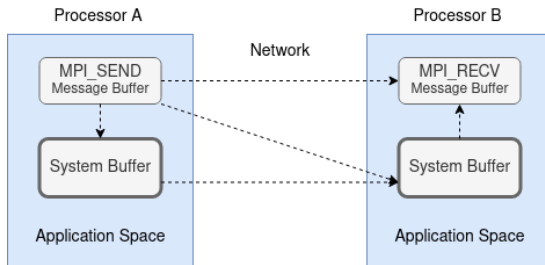


Figure 7: The standard mode either buffers the outgoing data or sends to the matching receiver - up to the MPI implementation to decide.

MPI_SEND

MPI_SEND(buf, count, datatype, dest, tag, comm)

IN **buf** *starting address of buffer (choice)*

IN **count** *number of entries in buffer (non-negative integer)*

IN **datatype** *datatype of elements in send buffer (handle)*

IN **dest** *rank of destination(integer)*

IN **tag** *message tag (integer)*

IN **comm** *communicator (handle)*

C Binding

- Blocking operations implies that neither the send buffer nor the argument list can be modified until the buffer is safely stored somewhere.
- System buffer may be outside the application space.
- How system buffer would be chosen is up to particular MPI implementations.
- Eager protocol - small messages.
- Rendezvous protocol - large messages.
- It may complete before the matching RECV is posted.

MPI_BSEND (Buffer mode)

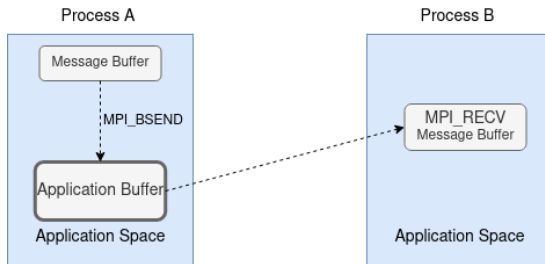


Figure 8: The buffer mode buffers the message to the application buffer provided by the user.

- Users are responsible to provide enough space for buffer.
- Only one buffer is allowed in a process at a time.
- The implementation varies.
- Be cautious to use this call.

MPI_SSEND (Synchronous mode)

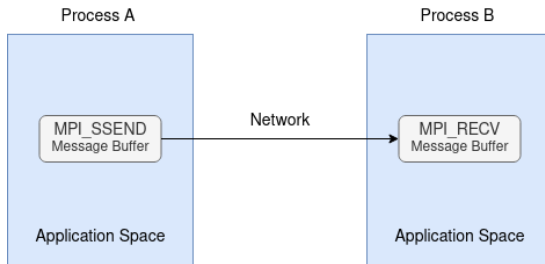


Figure 9: The communication does not complete at either end before both processes rendezvous at the communication.

- MPI_SSEND

- ▶ Completes only if a matching receive is posted, and the receiver has started to receive the message.
- ▶ Implies the sender has to hold until a matching receive is posted, and the receiver has started to receive the message before continuing.
- ▶ Ensures synchronisation: both processes have reached a certain point.
- ▶ Counter-intuitively, this call has no implication for buffering.

- Programmers should treat all sends synchronous, i.e. not rely on buffering to complete.

An erroneous example: Deadlock

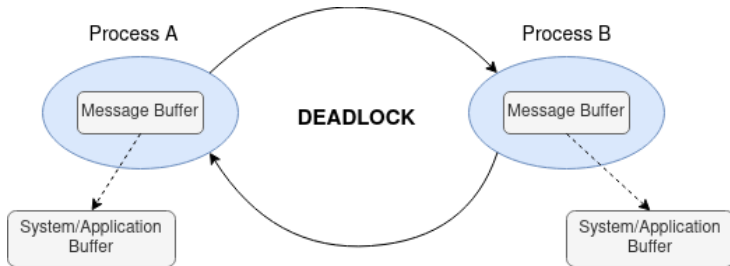


Figure 10: The message sent by each process has to be copied out before the send operation returns and the receive operation starts. For the program to complete, it is necessary that at least one of the two messages sent be buffered. Thus, this program can succeed only if the communication system can buffer at least *count* words of data. [?]

- RECV is asymmetric to SEND in MPI.
 - ▶ A receiver may accept messages from an arbitrary sender.
 - ▶ The length of the RECV buffer can be equal to or greater than the length of the RECV message.
 - ▶ MPI send and receive is a "push" mechanism.
- There is only one blocking RECV procedure but matches any of the send modes.

MPI_RECV

MPI_RECV(buf, count, datatype, source, tag, comm, status)

IN **buf** *starting address of buffer (choice)*

IN **count** *number of entries in buffer (non-negative integer)*

IN **datatype** *datatype of elements in send buffer (handle)*

IN **source** *rank of source or MPI_ANY_SOURCE (integer)*

IN **tag** *message tag or MPI_ANY_TAG (integer)*

IN **comm** *communicator (handle)*

OUT **status** *status object (status)*

C Binding

- Status in C: is a structure that contains three field named MPI_Source, MPI_TAG, MPI_ERROR
- Status in Fortran: is an array of integers.

Blocking Send and RECV

See the complementary code for details.

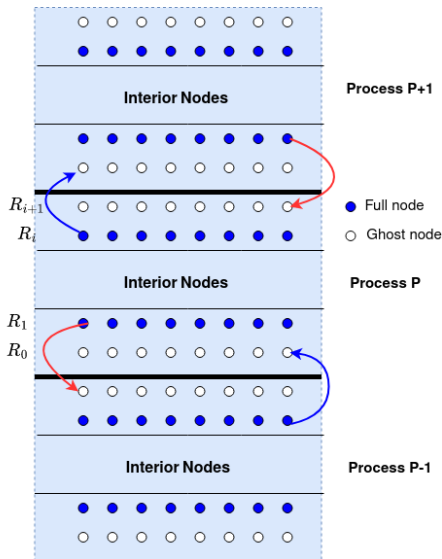


Figure 11: The communication of ghost nodes between processes.

In `laplace_mpi_blocking.c`, complete the communication by writing blocking send and receive for top row of full nodes using *`MPI_SEND`* and *`MPI_RECV`*.

- Network speed is much slower than floating-point operations
- Time spent on waiting for completing the matching SEND and RECV can be used for computation.

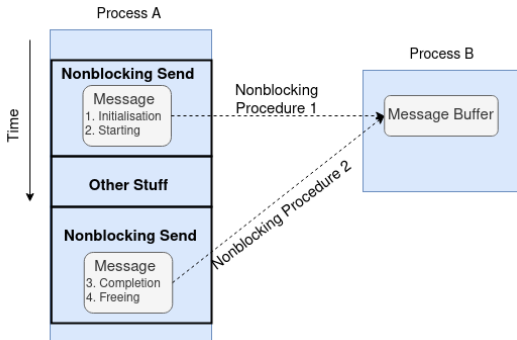


Figure 12: The nonblocking send is split into two MPI procedures.

Notic

MPI_ISEND

MPI_ISEND(buf, count, datatype, dest, tag, comm)

IN **buf** *starting address of buffer (choice)*

IN **count** *number of entries in buffer (non-negative integer)*

IN **datatype** *datatype of elements in send buffer (handle)*

IN **dest** *rank of destination(integer)*

IN **tag** *message tag (integer)*

IN **comm** *communicator (handle)*

OUT **request** *communication request(handle)*

C Binding

- The nonblocking send operation has the same communication modes as the blocking version.
- The arguments are the same as for MPI_Send with the addition of a handle *request*.
 - ▶ The *request* is an opaque object to identify communication operations and match the operation that initiates the communication with the operation that terminates it.
 - ▶ That is, the *request* handle is used in the nonblocking procedure 2.

MPI_WAIT

MPI_WAIT(**request**, **status**)

INOUT **request** *communication request (handle)*

OUT **status** *status object (status)*

MPI_WAITALL

MPI_WAITALL(**count**, **array_of_requests**, **array_of_statuses**)

IN **count** *list length (non-negative integer)*

INOUT **request** *communication request (handle)*

OUT **status** *status object (status)*

- MPI_WAIT returns when the operation identified by *request* is complete.
- MPI_WAITALL returns when all communication operations in the list are complete.
- What if we just want to check if a communication operation is complete rather than waiting for its completion?

MPI_TEST

int MPI_TEST(**request**, **flag**, **status**)

INOUT **request** *communication request (handle)*

OUT **flag** *true if operation completed (logical)*

OUT **status** *status object (status)*

MPI_TESTALL

MPI_TESTALL(**count**, **array_of_requests**, **flag**, **array_of_statuses**)

IN **count** *list length (non-negative integer)*

INOUT **request** *communication request (handle)*

OUT **flag** *true if all of the operations are complete (logical)*

OUT **status** *status object (status)*

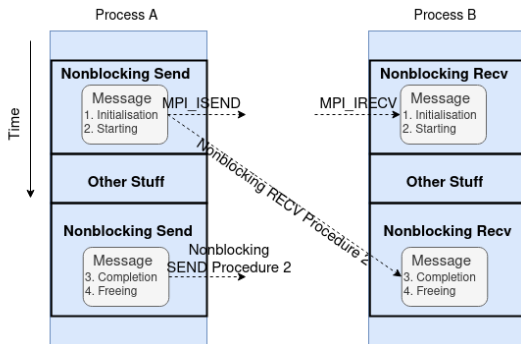


Figure 13: Nonblocking RECV may be posted before the matching send is initiated. Its completion does not indicate that the matching send operation has completed.

MPI_IRecv [?]

MPI_IRecv(**bug**, **count**, **datatype**, **source**, **tag**, **comm**, **request**)

OUT **buf** *starting address of buffer (choice)*

IN **count** *number of entries in buffer (non-negative integer)*

IN **datatype** *datatype of elements in send buffer (handle)*

IN **source** *rank of destination(integer)*

IN **tag** *message tag (integer)*

IN **comm** *communicator (handle)*

OUT **request** *communication request(handle)*

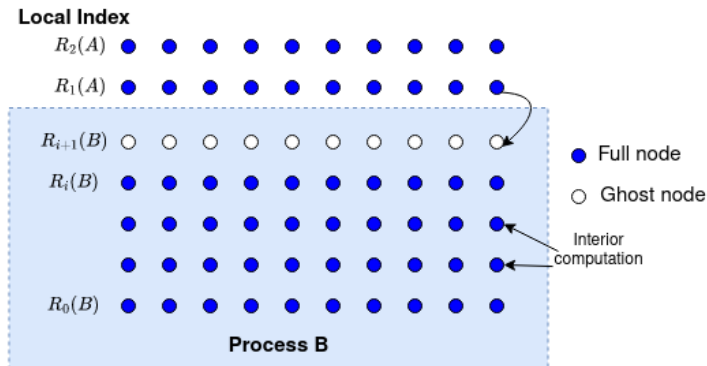
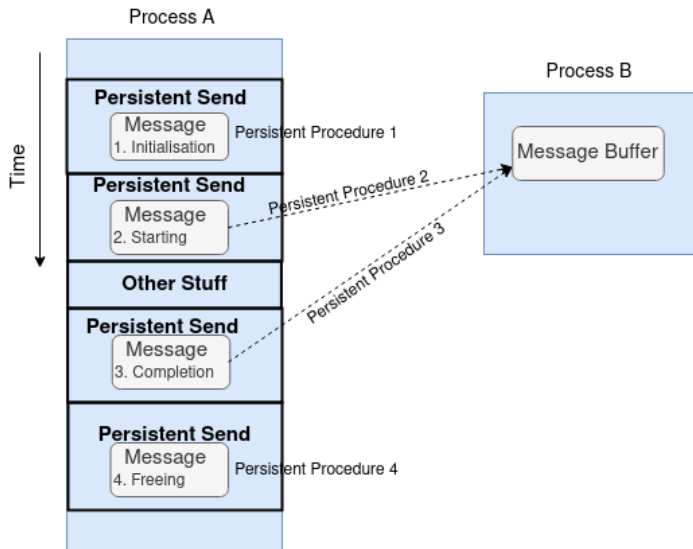


Figure 14: Jacobi method can perform on interior nodes independently without relying on communications.

See the complementary code for details.

Rewrite the nonblocking send and receive such that it waits until the completion of both top and bottom rows of ghost nodes.

Topic 1C: Persistent Communication



- A separate procedure for each of four stages.
- The same argument list is used in every single iteration and cumulatively creates overhead when get passed to communication controller repeatedly.
- Persistent communication allows the argument list to bind with the request once.

MPI_SEND_INIT

MPI_SEND_INIT(**buf**, **count**, **datatype**, **dest**, **tag**, **comm**, **request**)

IN **buf** *starting address of buffer (choice)*

IN **count** *number of entries in buffer (non-negative integer)*

IN **datatype** *datatype of elements in send buffer (handle)*

IN **dest** *rank of destination(integer)*

IN **tag** *message tag (integer)*

IN **comm** *communicator (handle)*

OUT **request** *communication request(handle)*

C Binding

MPI_START

MPI_START(**request**)

OUT **request** *communication request(handle)*

C Binding

MPI_STARTALL

MPI_STARTALL(**count**, **array_of_request**)

IN **count** *list length (non-negative integer)*

OUT **request** *communication request(handle)*

C Binding

- MPI_Send_INIT creates a persistent communication request, which involves no communication.
 - ▶ Communication that uses the persistent communication request is started by the procedure MPI_Start.

Persistent Send Procedure 3

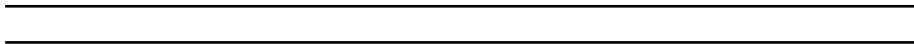
- This is a completion procedure, so we will use `MPI_WAIT`, `MPI_TEST` or their variations.

MPI_REQUEST_FREE

MPI_REQUEST_FREE(**request**)

OUT **request** *communication request(handle)*

C Binding



Refactor the nonblocking communication code to use persistence communication.

Showcase: 2D unstructured grid

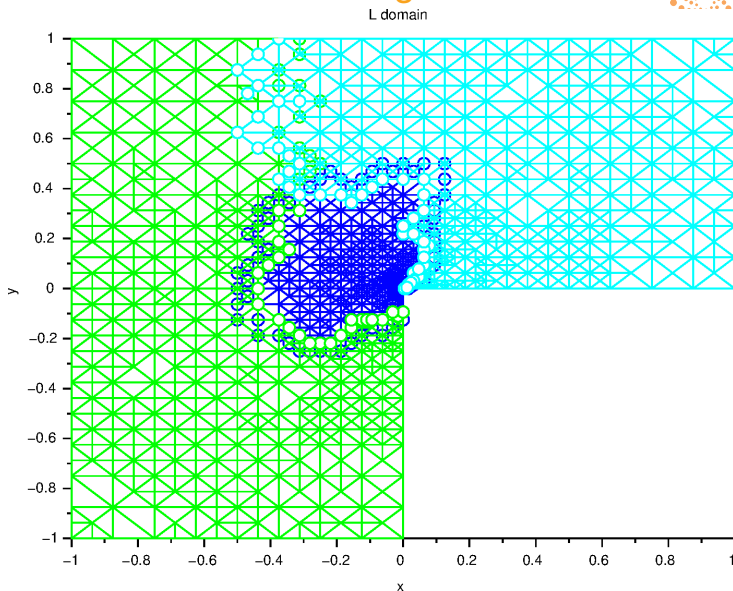


Figure 15: 2D adaptively refine grid distributed over three processors [?]. 🔍 ↻ 🔗

Topic 2: Collective Communication

Topic 2: Collective Communication

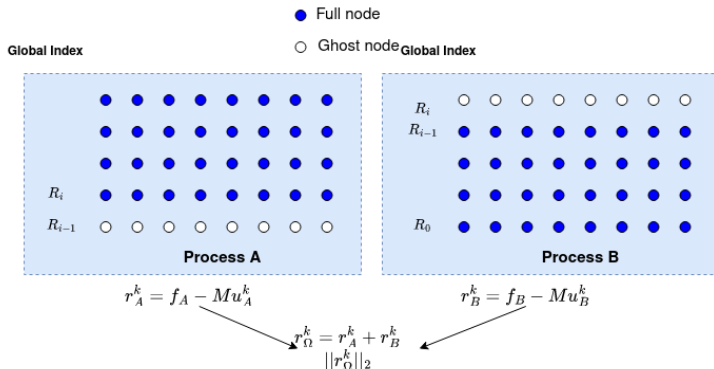


Figure 16: Collecting local residuals from each individual process to calculate the global residual.

Definition

Collective communication is defined as communication that involves a group or groups of processes.

- **All to All:** MPI_ALLREDUCE, MPI_BARRIER, MPI_ALLREDUCE_INIT, etc.
- **All to One:** MPI_GATHER, MPI_REDUCE, etc.
- **One to All:** MPI_SCATTER, MPI_BCAST, etc.
- **Other:** MPI_SCAN

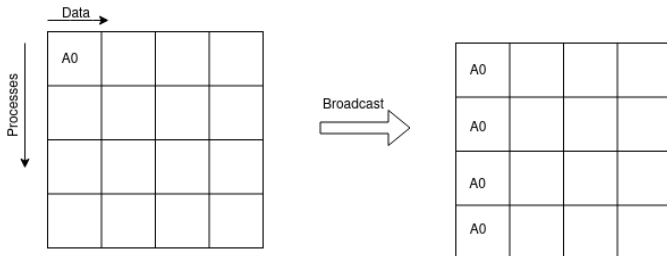


Figure 17: Each row of boxes represents data locations in one process. Initially just the first process contains the data A_0 , but after the broadcast all processes contain it [?].

MPI_BCAST

MPI_BCAST(**buffer**, **count**, **datatype**, **root**, **comm**)

INOUT **buffer** *starting address of buffer (choice)*

IN **count** *number of entries in buffer (non-negative integer)*

IN **datatype** *datatype of buffer (handle)*

IN **root** *rank of broadcast root (integer)*

IN **comm** *communicator (handle)*

C Binding

- All processes in the communicator must be involved,
- Count must be equal between the root and all other processes,
- Comm can be an inter-communicator,
- The order of execution of the collective operation should prevent deadlocking

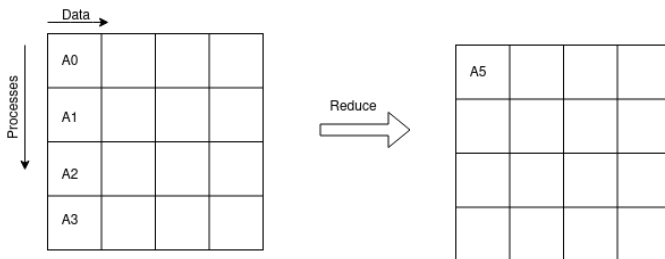


Figure 18: A global reduce operation performs across all members of the group and returns the result of the reduction to one member of the group [?].

MPI_REDUCE

MPI_REDUCE(**sendbuf**, **recvbuf**, **count**, **datatype**, **op**, **root**, **comm**)

IN **sendbuf** *address of send buffer (choice)*

OUT **recvbuf** *address of receive buffer (choice, significant only at root)*

IN **count** *number of elements in send buffer (non-negative integer)*

IN **datatype** *datatype of elements of send buffer (handle)*

IN **op** *reduce operation (handle)*

IN **root** *rank of root process (integer)*

IN **comm** *communicator (handle)*

Reduce the residuals to one process



- MPI_MAX: maximum
- MPI_MIN: minimum
- MPI_SUM: sum
- MPI_PROD: product
- MPI_MAXLOC: max value and location
- MPI_MINLOC: min value and location
- and a few more bit-wise operators.

User-defined reduction operation is also possible.

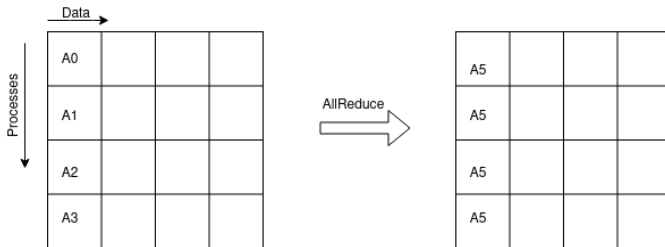


Figure 19: $\text{MPI_ALLREDUCE} = \text{MPI_REDUCE} + \text{MPI_BCAST}$.

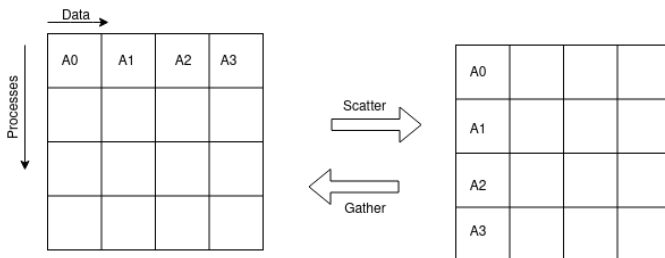


Figure 20: Gather: Each process contains the data A_i and sends to the root process. The root process receives the messages and stores them in the rank order[?].

MPI_Gather

MPI_Gather(**sendbuf**, **sendcount**, **sendtype**, **recvbug**, **recvcount**, **recvtype**, **root**, **comm**)

IN **sendbuf** *address of send buffer (choice)*

IN **sendcount** *number of elements in send buffer (non-negative integer)*

IN **sendtype** *datatype of send buffer elements (handle)*

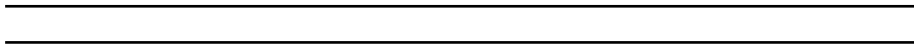
OUT **recvbuf** *address of receive buffer (choice, significant only at root)*

IN **recvcount** *number of elements for any single receive*

IN **recvtype** *datatype of recv buffer elements (handle, significant only at root)*

IN **root** *rank of root process (integer)*

IN **comm** *communicator (handle)*



In a floating-point system $\mathcal{F}$, the multiplication and addition operations are not associative. That is

$$(a + b) + c \neq a + (b + c) \quad \text{for } a, b, c \in \mathcal{F}.$$

Implying that the order of the operands may be important for reproducibility.

Replace the *MPI_REDUCE* call with *MPI_GATHER* to collect residuals to a root process in rank order. Then calculate the total residual from the root process.

- MPI_REDUCE, MPI_ALLREDUCE, MPI_GATHER, MPI_SCATTER, MPI_BCAST are also blocking procedures.
- There are non-blocking versions of those procedures following the same definition of the nonblocking operation that we discussed before.
- There also are persistent versions of those procedures following the same definition of the persistent operation that we discussed before.

Topic 3: One-sided Communication

Definition

One-sided communication aka. Remote Memory Access (RMA) reads, writes, updates and swaps data at a remote process without a matching receive or send.

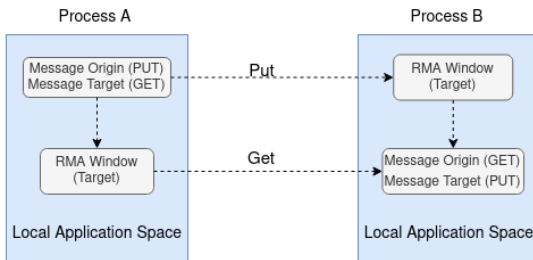


Figure 21: Transfer data from Origin to Target by One-sided Put and Get respectively.

Why one-sided communication?

- Asymmetry between regular SEND and RECV.
 - ▶ Message parameters are all available on one side.
- Avoids distributing transfer parameters.
 - ▶ By creating a window object that is accessible to processes at remote.

MPI_WIN_CREATE

MPI_WIN_CREATE(**base**, **size**, **disp_unit**, **info**, **comm**, **win**)

IN **base** *initial address of window (choice)*

IN **size** *size of window in bytes (non-negative integer)*

IN **disp_unit** *local unit size for displacements, in bytes (positive integer)*

IN **info** *info argument (handle)*

IN **comm** *intra-communicator (handle)*

OUT **window** *window object (handle)*

C Binding

- Creating local RMA window is a collective call - it must be called on every process in the group.
- This call does not allocate memory. The local memory needs to be allocated separately to be used in RMA operation.

Communicate ghost row through MPI_PUT

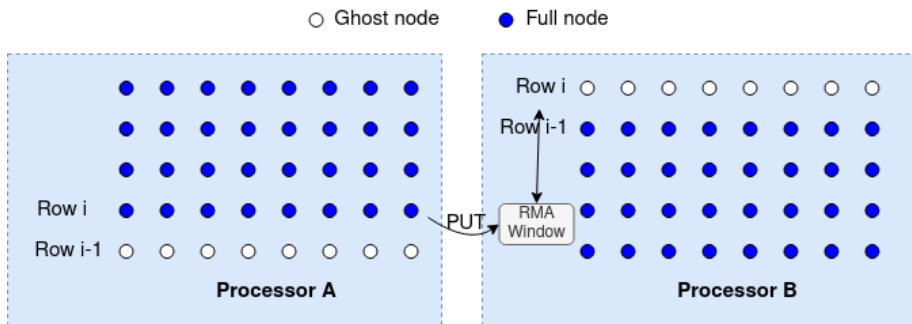


Figure 22: The i th row of full nodes in A is put into the local RMA window in B .

MPI_PUT

MPI_PUT (**origin_addr**, **origin_count**, **origin_datatype**, **target_rank**, **target_disp**, **target_count**, **target_datatype**, **win**)

IN **origin_addr** *initial address of origin buffer (choice)*

IN **origin_count** *number of entries in origin buffer (non-negative integer)*

IN **origin_datatype** *datatype of each entry in origin buffer (handle)*

IN **target_rank** *rank of target (non-negative integer)*

IN **target_disp** *displacement from start of window to target buffer (non-negative integer)*

IN **target_count** *number of entries in target buffer (non-negative integer)*

IN **target_datatype** *datatype of each entry in target buffer (handle)*

OUT **window** *window object (handle)*

C Binding

MPI_PUT: argument list explained

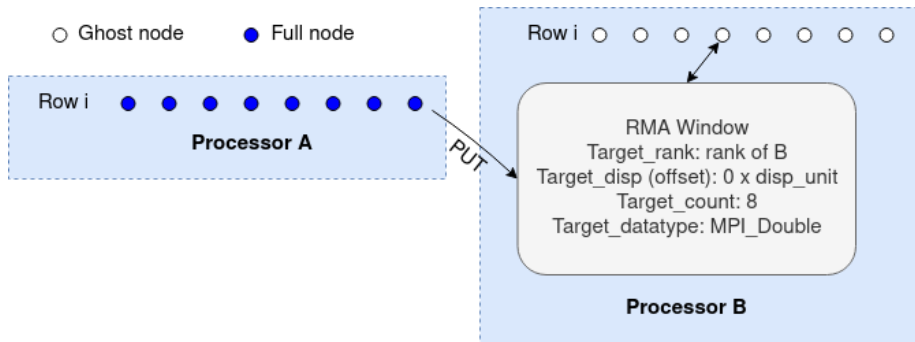


Figure 23: The i th row of full nodes in A is put directly into the local RMA window in B .

- target_disp is determined by the disp_unit specified in MPI_WIN_CREATE.
- Multiple MPI_PUT operations can be performed on a RMA window.
- MPI_PUT's behaviour is nonblocking alike a combination of MPI_ISEND and MPI_IRecv.
 - ▶ It needs another MPI call to complete.

MPI_WIN_FENCE

MPI_WIN_FENCE (**assert**, **win**)

IN **assert** *program assertion (integer)*

IN **win** *window object (handle)*

C Binding

Assertions

The *assert* flag is used to provide assertions on the context of the call that may be used to optimise performance.

- MPI_WIN_FENCE not only completes but also starts an *access epoch* at an origin process and an *exposure epoch* at an target process.
 - ▶ Access/Exposure epochs is the time when the local window is available for access.
 - ▶ "Switch" between origin and target.

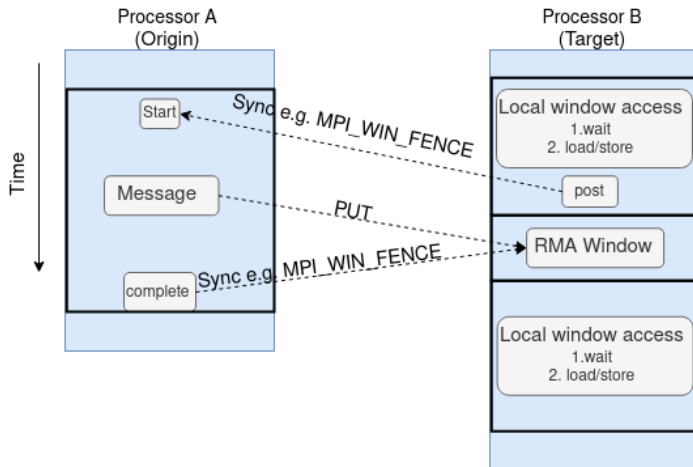


Figure 24: Active target communication where MPI_WIN_FENCE is used for synchronisation [?].

One-sided communication



Transfer both top and bottom row of ghost nodes in one RMA window by carefully specifying the `target_disp` argument in `MPI_PUT`.

Topic 5: MPI Profiling

PMPI_Prefix [?]

PMPI_prefix defines an alternate entry point name for each MPI function.

Example: MPI_SEND, PMPI_SEND.

- The functions with PMPI_ prefix have the same functionality as the MPI version.
 - ▶ This allows profiling tools to intercept applications' MPI function calls and do surrounding profiling work.

User's MPI call $\xRightarrow{\text{intercepted}}$ Profiler Library $\xRightarrow{\text{internal call}}$ PMPI call

A simple profiler implementation [?]

When this profiler library is linked it will intercept `MPI_SEND` call from users' application to count elapsed time and the amount of data.

- mpiP is a light-weight profiling library for MPI applications.
 - ▶ Build on top of PMPI.
 - ▶ Measures the entire MPI application from MPI_INIT to MPI_FINALIZE
- mpiP 3.4 is available on Gadi by module load.

Using mpiP to measure the speedup for blocking, nonblocking and one-sided communication codes.

Topic 6: Basic MPI-IO

- Reasonable number of files under per parent directory to avoid lock contention - 10k-ish.
- Use striping file for parallel access for very large files.
- Calculate input/output data size. Read/write large size of data (>0.5 MB) if possible.

Courtesy of Kim

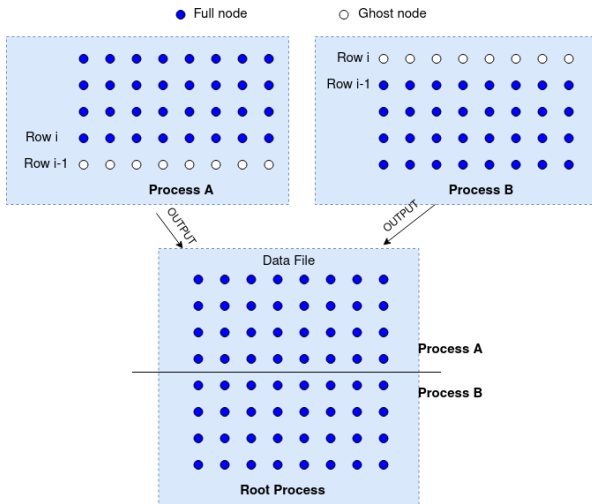


Figure 25: Output submeshes from their local processes to a single file.

- With Posix-IO,
 - ▶ each process writes data to individual files or
 - ▶ one process gathers all data and output a single file.
- With MPI-IO,
 - ▶ multiple processes concurrently write to a single file.

Of course, the parallel IO capability won't be possible without parallel file system support. On Gadi, it is our lustre file system.

- Open the file.
- Write to/read from the file.
- Close the file.

MPI_FILE_OPEN [?]

MPI_FILE_OPEN(**comm**, **filename**, **amode**, **info**, **fh**)

IN **comm** *initial communicator (handle)*

IN **filename** *name of file to open (string)*

IN **amode** *file access mode (integer)*

IN **info** *info object (handle)*

OUT **fh** *new file handle(handle)*

C Binding

- The call is a collective routine.
 - ▶ All process in the comm group must provide the same amode
 - ★ MPI_MODE_CREATE: create the file if it does not exist;
 - ★ MPI_MODE_WRONLY: write only;
 - ★ MPI_MODE_RDONLY: read only;
 - ★ and many more.

MPI_FILE_CLOSE [?]

MPI_FILE_CLOSE(**fh**)

INOUT **fh** *file handle (handle)*

C Binding

MPI_FILE_WRITE_AT [?]

MPI_FILE_WRITE_AT(fh, offset, buf, count, datatype, status)

INOUT **fh** *file handle (handle)*

IN **offset** *file offset (integer)*

OUT **buf** *initial address of buffer (choice)*

IN **count** *number of elements in buffer (integer)*

IN **datatype** *datatype of each buffer element (handle)*

OUT **status** *status object (status)*

C Binding

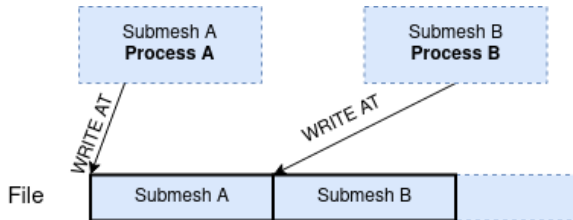
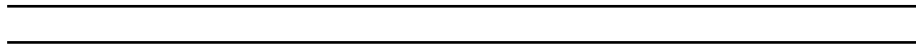


Figure 26: Each process writes at the offset position of the common file.

- The call is noncollective. The collective version is defined with _ALL suffix.
- The call writes unformatted binary file.



References I

Thank you!

Please help us improve!

https:
//anu.au1.qualtrics.com/jfe/form/SV_0BRRQvSz5oiwOqO